

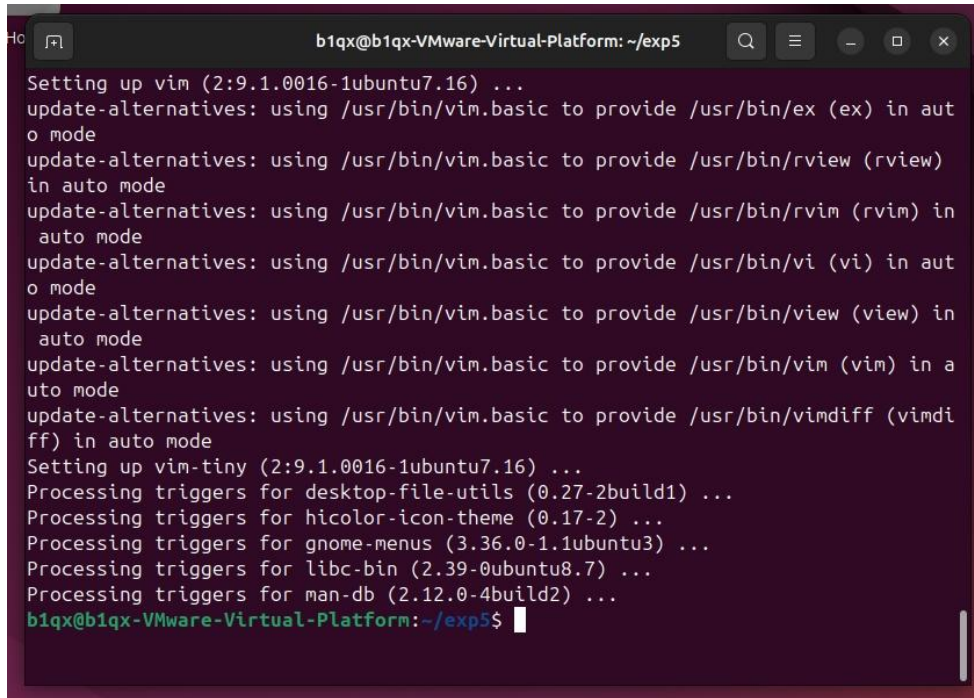
天津大学
智能与计算学部
网络空间安全学院

《软件安全》实验报告
实验五：AFL 模糊测试工具使用

姓名：b1qx

一. 实验环境

本实验在 VMware 虚拟机中完成, 操作系统为 Ubuntu 22.04 LTS 64 位。实验使用 AFL 2.52b 作为覆盖引导模糊测试工具, 使用 GCC、Make 和 afl-gcc 完成源码编译与插桩, 使用 GDB 对崩溃样本进行复现和定位, 并使用 Git、Vim 等工具完成源码获取与编辑。



```
b1qx@b1qx-VMware-Virtual-Platform: ~/exp5
Setting up vim (2:9.1.0016-1ubuntu7.16) ...
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/ex (ex) in aut
o mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/rview (rview)
in auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/rvim (rvim) in
auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/vi (vi) in aut
o mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/view (view) in
auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/vim (vim) in a
uto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/vimdiff (vimdi
ff) in auto mode
Setting up vim-tiny (2:9.1.0016-1ubuntu7.16) ...
Processing triggers for desktop-file-utils (0.27-2build1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for gnome-menus (3.36.0-1.1ubuntu3) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for man-db (2.12.0-4build2) ...
b1qx@b1qx-VMware-Virtual-Platform:~/exp5$
```

图 1 Ubuntu 22.04 中安装 GCC、Make、Git、GDB 和 Vim 等实验依赖

二. 实验目的

1. 下载、编译并验证 AFL 2.52b, 熟悉 afl-gcc、afl-fuzz 和 afl-showmap 等核心工具。
2. 基于给定的 C 语言示例程序完成编译期插桩, 理解插桩信息在代码覆盖率反馈中的作用。
3. 创建初始种子并运行覆盖引导模糊测试, 观察路径增长、执行速度和崩溃发现过程。
4. 提取并复现 AFL 发现的 crash 样本, 使用 GDB 定位崩溃信号、调用栈和触发输入。
5. 理解 AFL 计算边覆盖率的基本原理, 以及位翻转、字节翻转、算术变异、havoc 和 splice 等样本变异方法。

三. 实验原理

3.1 模糊测试基本原理

模糊测试是一种动态漏洞挖掘方法。测试工具不断生成或变异输入，将输入提交给目标程序运行，同时监测程序是否出现崩溃、超时或异常行为。按照测试数据的来源，模糊测试通常分为基于生成的模糊测试和基于变异的模糊测试。本实验中的 AFL 从合法种子 hello 出发，通过多种变异策略自动生成新样本，属于基于变异的模糊测试。

3.2 AFL 覆盖引导机制

AFL 在编译目标程序时插入轻量级探针。程序每执行到一个插桩位置，探针会根据当前基本块位置和前一个基本块位置计算边信息，并把结果记录到共享内存位图中。AFL 比较每个新输入产生的位图：当输入触发新的控制流边或使计数落入新的区间时，该输入会被保留到队列，作为后续变异的基础；没有带来新覆盖的输入通常不会继续保留。通过“执行—反馈—保留—再变异”的循环，AFL 能逐步进入更深的分支。

3.3 样本变异方法

AFL 的确定性阶段会进行 1 位、2 位、4 位翻转，整字节翻转，算术加减以及特殊整数替换；随后进入 havoc 阶段，随机组合多种变异；在条件合适时还会使用 splice 将两个队列样本拼接后继续变异。本实验最终 crash 文件名中出现 op:arith8、pos:7、val:+13，表明该崩溃样本由 8 位算术变异得到，变异位置为第 7 个偏移字节。

3.4 示例程序触发条件

示例程序读取输入文件中的字符串，并依次比较前 8 个字符。当输入满足 d、e、a、d、b、e、e、f，即字符串 deadbeef 时，程序调用 abort() 主动产生 SIGABRT。连续嵌套判断形成逐层深入的执行路径，适合用于观察覆盖反馈如何引导 AFL 逐字节逼近目标输入。该示例的崩溃点是人为设置的测试触发点，主要用于验证 fuzzing 流程，并不等同于真实的可利用内存破坏漏洞。

四. 实验步骤

4.1 下载并编译 AFL

首先创建实验目录，安装编译、调试和版本控制工具。随后从归档仓库获取 AFL 2.52b 源码，在源码目录执行 make 和 make test_build。test_build 会使用 AFL 编译包装器构建测试程序，并通过 afl-showmap 检查不同输入是否产生不同的覆盖信息。

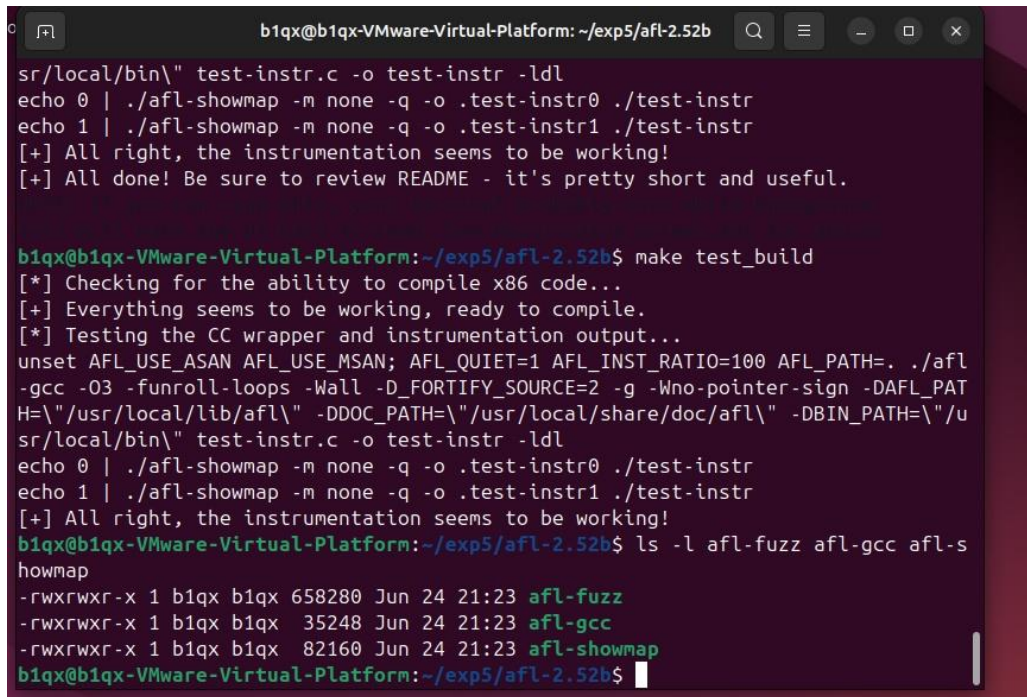
```
mkdir -p ~/exp5
```

```

cd ~/exp5
git clone https://github.com/Fuzzers-Archive/afl-2.52b.git
cd afl-2.52b
make
make test_build
ls -l afl-fuzz afl-gcc afl-showmap

```

编译结果显示 “All right, the instrumentation seems to be working!”; 同时 afl-fuzz、afl-gcc 和 afl-showmap 均成功生成,说明 AFL 编译与基础插桩测试通过。



```

b1qx@b1qx-VMware-Virtual-Platform: ~/exp5/afl-2.52b
sr/local/bin\" test-instr.c -o test-instr -ldl
echo 0 | ./afl-showmap -m none -q -o .test-instr0 ./test-instr
echo 1 | ./afl-showmap -m none -q -o .test-instr1 ./test-instr
[+] All right, the instrumentation seems to be working!
[+] All done! Be sure to review README - it's pretty short and useful.

b1qx@b1qx-VMware-Virtual-Platform:~/exp5/afl-2.52b$ make test_build
[*] Checking for the ability to compile x86 code...
[+] Everything seems to be working, ready to compile.
[*] Testing the CC wrapper and instrumentation output...
unset AFL_USE_ASAN AFL_USE_MSAN; AFL_QUIET=1 AFL_INST_RATIO=100 AFL_PATH=. ./afl
-gcc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH=
"/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/u
sr/local/bin\" test-instr.c -o test-instr -ldl
echo 0 | ./afl-showmap -m none -q -o .test-instr0 ./test-instr
echo 1 | ./afl-showmap -m none -q -o .test-instr1 ./test-instr
[+] All right, the instrumentation seems to be working!
b1qx@b1qx-VMware-Virtual-Platform:~/exp5/afl-2.52b$ ls -l afl-fuzz afl-gcc afl-s
howmap
-rwxrwxr-x 1 b1qx b1qx 658280 Jun 24 21:23 afl-fuzz
-rwxrwxr-x 1 b1qx b1qx 35248 Jun 24 21:23 afl-gcc
-rwxrwxr-x 1 b1qx b1qx 82160 Jun 24 21:23 afl-showmap
b1qx@b1qx-VMware-Virtual-Platform:~/exp5/afl-2.52b$

```

图 2 AFL 编译、test_build 验证及核心工具生成成功

4.2 编写并插桩编译目标程序

将课件中的示例程序保存为 test.c。程序既支持从命令行指定的文件读取输入,也支持从标准输入读取。为了便于 GDB 显示源码行号,编译时加入 -g;使用 -O0 关闭优化,使调试信息与源码结构保持一致。

```

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
    char ptr[20];

    if (argc > 1) {
        FILE *fp = fopen(argv[1], "r");
        if (fp == NULL) {
            return 1;
        }
        fgets(ptr, sizeof(ptr), fp);
    }
}

```

```

        fclose(fp);
    } else {
        fgets(ptr, sizeof(ptr), stdin);
    }

    printf("%s", ptr);

    if (ptr[0] == 'd') {
        if (ptr[1] == 'e') {
            if (ptr[2] == 'a') {
                if (ptr[3] == 'd') {
                    if (ptr[4] == 'b') {
                        if (ptr[5] == 'e') {
                            if (ptr[6] == 'e') {
                                if (ptr[7] == 'f') {
                                    abort();
                                } else {
                                    printf("%c", ptr[7]);
                                }
                            } else {
                                printf("%c", ptr[6]);
                            }
                        } else {
                            printf("%c", ptr[5]);
                        }
                    } else {
                        printf("%c", ptr[4]);
                    }
                } else {
                    printf("%c", ptr[3]);
                }
            } else {
                printf("%c", ptr[2]);
            }
        } else {
            printf("%c", ptr[1]);
        }
    } else {
        printf("%c", ptr[0]);
    }

    return 0;
}

./afl-2.52b/afl-gcc -g -O0 -o test test.c

```

编译器给出了未检查 `fgets` 返回值的警告，但未产生编译错误。AFL 输出 “Instrumented 19 locations”，表示在目标程序中插入了 19 个覆盖率记录位置，插桩编译成功。

```
]
b1qx@b1qx-VMware-Virtual-Platform:~/exp5$ ./afl-2.52b/afl-gcc -g -O0 -o test tes
t.c
afl-cc 2.52b by <lcamtuf@google.com>
test.c: In function 'main':
test.c:12:9: warning: ignoring return value of 'fgets' declared with attribute '
warn_unused_result' [-Wunused-result]
   12 |         fgets(ptr, sizeof(ptr), fp);
      |         ^
test.c:15:9: warning: ignoring return value of 'fgets' declared with attribute '
warn_unused_result' [-Wunused-result]
   15 |         fgets(ptr, sizeof(ptr), stdin);
      |         ^
afl-as 2.52b by <lcamtuf@google.com>
[+] Instrumented 19 locations (64-bit, non-hardened mode, ratio 100%).
b1qx@b1qx-VMware-Virtual-Platform:~/exp5$
```

图 3 使用 afl-gcc 插桩编译目标程序，共插入 19 个监测位置

4.3 准备初始种子

删除可能存在的旧输入和输出目录，新建 `in` 目录，并以字符串 `hello` 创建初始种子。该种子能够被程序正常读取，是 AFL 变异的起点。

```
cd ~/exp5
rm -rf in out
mkdir in
echo hello > in/seed
ls -l in
cat in/seed
```

操作过程中曾把 `seed` 误输为 `need`，并把目录误输为 `on`，导致程序没有读取到预期文件。确认实际文件名为 `in/seed` 后，使用正确路径继续实验。

```
b1qx@b1qx-VMware-Virtual-Platform:~/exp5$ rm -rf in out
mkdir in
echo hello > in/seed
b1qx@b1qx-VMware-Virtual-Platform:~/exp5$ ls -l in
cat in/seed
total 4
-rw-rw-r-- 1 b1qx b1qx 6 Jun 24 21:33 seed
hello
b1qx@b1qx-VMware-Virtual-Platform:~/exp5$ ./test on/need
b1qx@b1qx-VMware-Virtual-Platform:~/exp5$ ./test in/need
```

图 4 创建种子文件并检查文件内容，同时排查输入路径拼写错误

4.4 启动 AFL 模糊测试

首先将 `kernel.core_pattern` 设置为 `core`，保证程序崩溃时的系统行为符合 AFL 检查要求。由于实验运行在虚拟机中，使用 `AFL_SKIP_CPUFREQ=1` 跳过 CPU 调频策略检查。启动命令中的 `-i` 指定种子目录，`-o` 指定输出目录，`@@` 会

在每次执行时被替换为当前变异样本的文件路径。

```
sudo sysctl -w kernel.core_pattern=core
AFL_SKIP_CPUFREQ=1 ./afl-2.52b/afl-fuzz -i in -o out -- ./test @@
```

第一次启动时，AFL 提示终端不足 80×25，无法显示运行界面。将终端最大化并缩小字体后，界面能够正常显示。

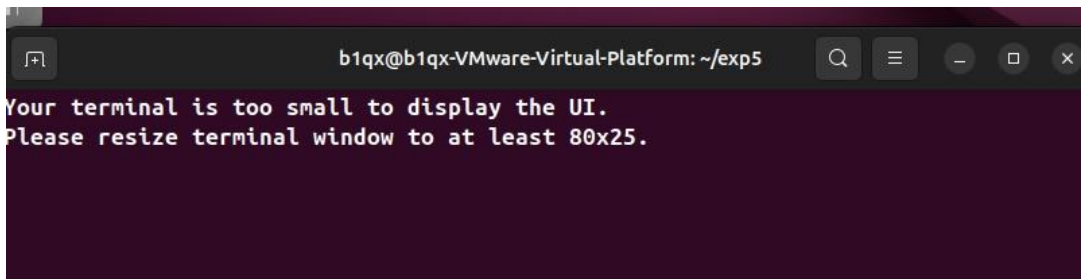


图 5 AFL 提示终端尺寸不足，需要至少 80 列×25 行

4.5 观察模糊测试过程

AFL 启动后持续变异队列样本。运行约 1 分 30 秒时，已经执行约 5.4 万个测试用例，发现 5 条路径，但 unique crashes 仍为 0，说明 AFL 已经进入部分嵌套分支，但尚未生成完整的 deadbeef。

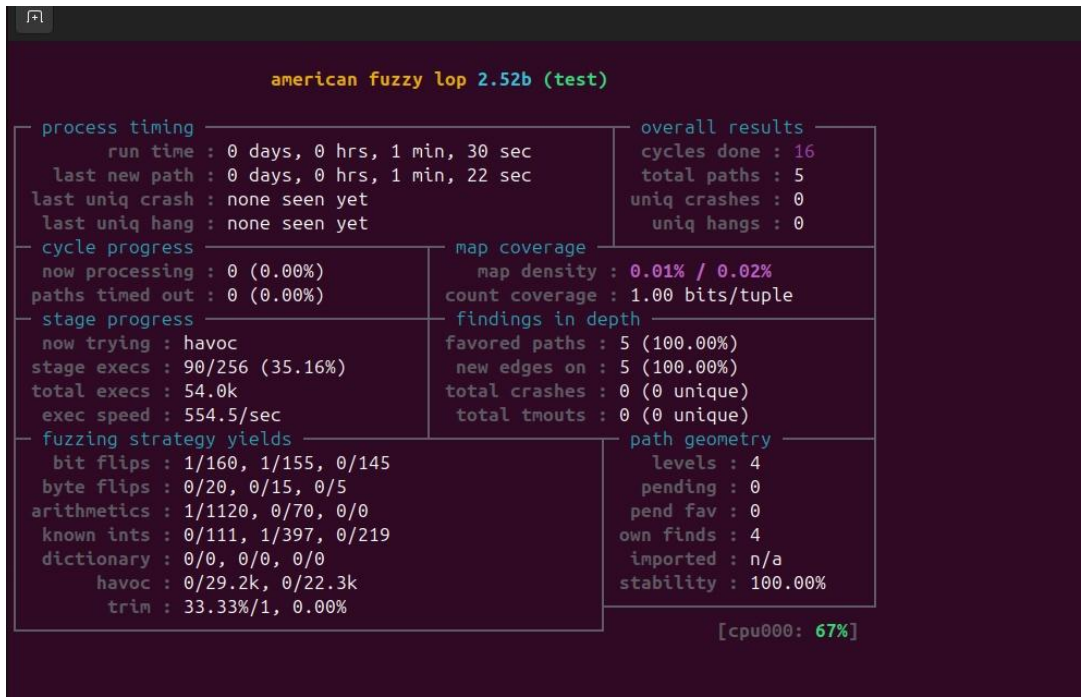


图 6 AFL 运行中：已发现 5 条路径，尚未发现崩溃

继续运行到约 3 分 45 秒时，AFL 总执行次数约为 13.3 万次，执行速度约为 609.2 次/秒，共发现 8 条路径，并记录 1 个唯一崩溃。界面中的 stability 为 100%，说明相同输入产生的覆盖结果稳定。此时保存运行界面并按 Ctrl+C 停止测试。

american fuzzy lop 2.52b (test)			
process timing		overall results	
run time : 0 days, 0 hrs, 3 min, 45 sec		cycles done : 18	
last new path : 0 days, 0 hrs, 2 min, 10 sec		total paths : 8	
last uniq crash : 0 days, 0 hrs, 1 min, 20 sec		uniq crashes : 1	
last uniq hang : none seen yet		uniq hangs : 0	
cycle progress		map coverage	
now processing : 6 (75.00%)		map density : 0.02% / 0.03%	
paths timed out : 0 (0.00%)		count coverage : 1.00 bits/tuple	
stage progress		findings in depth	
now trying : splice 9		favored paths : 8 (100.00%)	
stage execs : 72/256 (28.12%)		new edges on : 8 (100.00%)	
total execs : 133k		total crashes : 1 (1 unique)	
exec speed : 609.2/sec		total tmouts : 0 (0 unique)	
fuzzing strategy yields		path geometry	
bit flips : 3/352, 1/344, 0/328		levels : 6	
byte flips : 0/44, 0/36, 0/20		pending : 0	
arithmetics : 2/2461, 0/88, 0/0		pend fav : 0	
known ints : 0/245, 1/976, 0/879		own finds : 7	
dictionary : 0/0, 0/0, 0/0		imported : n/a	
havoc : 0/79.9k, 1/48.1k		stability : 100.00%	
trim : 12.50%/5, 0.00%			
[cpu000: 74%]			

图 7 AFL 成功发现 1 个 unique crash

4.6 提取并复现崩溃样本

AFL 将导致程序崩溃的输入保存在 out/crashes 目录。通过 find 命令自动获取第一个非 README 的样本，并把路径保存到 CRASH 变量中。

```
CRASH=$(find out/crashes -type f ! -name README.txt | head -n 1)
echo "$CRASH"
./test "$CRASH"
```

崩溃样本文件名为 id:000000,sig:06,src:000007,op:arith8,pos:7,val:+13。其中 sig:06 表示程序由 6 号信号 SIGABRT 终止；其余字段记录了父样本、变异类型、位置和数值。直接运行该样本可稳定复现异常。

4.7 使用 GDB 定位崩溃原因

使用 GDB 带崩溃样本启动 test，执行 run 后程序收到 SIGABRT。GDB 首先停在 glibc 的 pthread_kill 实现中，这是 abort() 触发信号的系统调用链，并不表示系统库存在漏洞。

```
gdb -q --args ./test "$CRASH"
run
```



```

b1qx@b1qx-VMware-Virtual-Platform:~/exp5$ gdb -q --args ./test "$CRASH"
Reading symbols from ./test...
(gdb) run
Starting program: /home/b1qx/exp5/test out/crashes/id:000000,sig:06,src:000007,op:arith8,pos:7,val:+13

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGABRT, Aborted.
Download failed: Invalid argument. Continuing without source file ./nptl./nptl/pthread_kill.c.
__pthread_kill_implementation (no_tid=0, signo=6, threadid=<optimized out>) at ./nptl/pthread_kill.c:44
warning: 44 ./nptl/pthread_kill.c: No such file or directory
(gdb)

```

图 8 GDB 复现崩溃，程序收到 SIGABRT

执行 bt 查看调用栈，栈帧 #4 为 __GI_abort，栈帧 #5 为 main，位置为 test.c:28。调用栈证明 SIGABRT 最终由目标程序 main 函数中的 abort() 触发。

```

b1qx@b1qx-VMware-Virtual-Platform:~/exp5$ gdb -q --args ./test "$CRASH"
Reading symbols from ./test...
(gdb) run
Starting program: /home/b1qx/exp5/test out/crashes/id:000000,sig:06,src:000007,op:arith8,pos:7,val:+13

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGABRT, Aborted.
Download failed: Invalid argument. Continuing without source file ./nptl./nptl/pthread_kill.c.
__pthread_kill_implementation (no_tid=0, signo=6, threadid=<optimized out>) at ./nptl/pthread_kill.c:44
warning: 44 ./nptl/pthread_kill.c: No such file or directory
(gdb) bt
#0  __pthread_kill_implementation (no_tid=0, signo=6, threadid=<optimized out>) at ./nptl/pthread_kill.c:44
#1  __pthread_kill_internal (signo=6, threadid=<optimized out>) at ./nptl/pthread_kill.c:78
#2  __GI___pthread_kill (threadid=<optimized out>, signo=signo@entry=6) at ./nptl/pthread_kill.c:89
#3  0x00007ffff7c4527e in __GI_raise (sig=sig@entry=6) at ../sysdeps/posix/raise.c:26
#4  0x00007ffff7c288ff in __GI_abort () at ./stdlib/abort.c:79
#5  0x000055555555527d in main (argc=<optimized out>, argv=<optimized out>) at test.c:28
(gdb)

```

图 9 调用栈定位到 main 函数 test.c 第 28 行

切换到 frame 5 后，使用 list 23,31 查看源代码，能够看到第 28 行 abort()；使用 x/s ptr 查看局部缓冲区，内容为 deadbeef。由此确认 AFL 自动生成的输入完整满足 8 层字符比较条件，并触发预设崩溃点。

```

frame 5
list 23,31
x/s ptr

```

```

(gdb) bt
#0 __pthread_kill_implementation (no_tid=0, signo=6, threadid=<optimized out>) at ./nptl/pthread_kill.c:
#1 __pthread_kill_internal (signo=6, threadid=<optimized out>) at ./nptl/pthread_kill.c:78
#2 __GI___pthread_kill (threadid=<optimized out>, signo=signo@entry=6) at ./nptl/pthread_kill.c:89
#3 0x00007ffff7c4527e in __GI_raise (sig=sig@entry=6) at ../sysdeps/posix/raise.c:26
#4 0x00007ffff7c288ff in __GI_abort () at ./stdlib/abort.c:79
#5 0x000055555555527d in main (argc=<optimized out>, argv=<optimized out>) at test.c:28
(gdb) frame 5
#5 0x000055555555527d in main (argc=<optimized out>, argv=<optimized out>) at test.c:28
28      abort();
(gdb) list 23,31
23          if (ptr[3] == 'd') {
24              if (ptr[4] == 'b') {
25                  if (ptr[5] == 'e') {
26                      if (ptr[6] == 'e') {
27                          if (ptr[7] == 'f') {
28                              abort();
29                          } else {
30                              printf("%c", ptr[7]);
31                          }
(gdb) x/s ptr
0x7fffffffdd20: "deadbeef"
(gdb)

```

图 10 GDB 显示 abort() 源码位置及触发输入 deadbeef

五. 实验结果与分析

5.1 实验结果

AFL 2.52b 在 Ubuntu 22.04 虚拟机中成功完成编译和自检；目标程序经 afl-gcc 插桩后共插入 19 个监测位置。以 hello 为初始种子运行约 3 分 45 秒，AFL 执行约 13.3 万个样本，发现 8 条路径和 1 个唯一崩溃。崩溃样本在 GDB 中稳定产生 SIGABRT，调用栈定位到 test.c 第 28 行，缓冲区内容为 deadbeef。实验目标全部达到。

5.2 覆盖率反馈分析

示例程序采用连续嵌套判断，每多匹配一个字符，就会进入更深的控制流路径。初始种子 hello 在第一个字符处即失败，只能覆盖浅层路径。AFL 发现某次变异使首字符变为 d 后，会因为出现新边而保留该样本；后续样本依次匹配 de、dea、dead 等前缀时，也会不断产生新的覆盖并被加入队列。因此 AFL 不需要一次随机生成完整的 deadbeef，而是利用覆盖反馈逐层推进。界面中 total paths 从 5 增长到 8，反映了这一过程。

5.3 崩溃样本分析

崩溃样本名称中的 sig:06 与 GDB 显示的 SIGABRT 完全一致。op:arith8、pos:7、val:+13 表明最终一步使用 8 位算术变异修改了偏移 7 处的字节，使最后一个字符满足 f。GDB 的 x/s ptr 直接显示 deadbeef，从输入内容、文件名元数据、调用栈和源代码位置四个方面相互验证了分析结论。

5.4 可利用性说明

本实验目标程序在识别到特定字符串后主动调用 abort()，因此 AFL 检测到的是人为设置的异常终止点。该结果能够验证 AFL 的插桩、覆盖反馈、样本变

异、崩溃保存与复现流程，但不能据此认定存在可利用的缓冲区溢出或控制流劫持漏洞。真实漏洞挖掘中还需要结合越界访问、Use-After-Free、整数溢出等根因以及内存保护机制进一步判断可利用性。

六. 实验中遇到的问题及解决办法

1. **AFL 运行界面无法显示：**终端尺寸小于 80×25 时出现 “Your terminal is too small” 提示。通过最大化终端、进入全屏并缩小字体解决。
2. **虚拟机 CPU 频率检查：**虚拟机中无法可靠控制宿主机 CPU governor，启动时在命令前加入 `AFL_SKIP_CPUFREQ=1`，跳过频率检查。
3. **种子文件路径拼写错误：**曾输入 `on/need` 和 `in/need`，程序无法读取预期种子。使用 `ls -l in` 和 `cat in/seed` 核对目录及文件名后，改为 `./test in/seed`。
4. **fgets 返回值警告：**编译时提示忽略 fgets 返回值。该警告不影响本实验运行，但在实际程序中应检查 fgets 是否返回 NULL，避免输入失败后继续使用未初始化数据。
5. **GDB 提示系统库源码不存在：**GDB 显示 `pthread_kill.c: No such file or directory`，是因为本机没有对应 glibc 源码。通过 `bt` 切换到 main 栈帧即可查看自己的 `test.c`，不影响崩溃定位。

七. 心得体会

通过本次实验，我完整经历了从 AFL 环境搭建、目标程序插桩、种子准备、模糊测试运行到 crash 复现和 GDB 定位的全过程。以前对模糊测试的理解主要停留在“随机输入大量数据”，本次实验使我认识到 AFL 的关键并不是单纯增加随机次数，而是利用代码覆盖率作为反馈，保留能够进入新路径的样本，再在这些样本基础上继续变异。

示例程序的嵌套判断非常直观地体现了覆盖引导的价值。AFL 不需要直接猜中 `deadbeef`，而是通过新路径反馈逐步匹配更长前缀。最终 crash 文件名中的 `arith8` 变异信息、GDB 的 `SIGABRT` 调用栈、`test.c` 第 28 行的 `abort()` 和 `ptr` 中的 `deadbeef` 能够形成完整证据链，使我掌握了“发现异常—保存样本—稳定复现—定位根因”的基本漏洞挖掘流程。

同时，我也注意到发现 crash 并不等于发现了可利用漏洞。本实验中的 `abort()` 是预设触发点，真实软件中还需要判断异常是否由内存破坏、边界检查缺失或对象生命周期错误等问题引起，并进一步分析攻击者能否控制关键数据或程序控制流。本次实验为后续学习更复杂的模糊测试目标、语法感知变异和漏洞可利用性分析打下了基础。